



VIT BHOPAL UNIVERSITY

School of Computing Science and Engineering

Bhopal-Indore Highway, Kothrikalan, Sehore

Madhya Pradesh - 466114

REG.NO : 26BME10004

NAME : Fahim Ahmed

BRANCH : Core Mechanical

SEMESTER : Fall Semester 2026-27

SMART STUDENT PERFORMANCE ANALYZER

Comprehensive System Engineering, Architectural Design, and
Analytical Project Report

Author: Fahim Ahmed

Document Version: 1.0.0

Date: September 2026

1. EXECUTIVE SUMMARY

The **Smart Student Performance Analyzer** is an automated software architecture and data analytics engine developed in Python to optimize, streamline, and scale educational assessment workflows. Modern academic environments generate massive volumes of student assessment data spanning multiple subjects, terms, and evaluation criteria. However, educational institutions frequently rely on manual spreadsheets or legacy administrative tools that fail to provide real-time aggregate insights, predictive performance tiering, or individualized diagnostic metrics.

This project addresses critical gaps in academic evaluation systems by providing a modular, object-oriented software solution capable of ingesting raw multi-subject assessment datasets, standardizing and cleaning data streams, dynamically computing parametric statistical

aggregates, categorizing students into actionable pedagogical intervention tiers, and automatically rendering data visualizations alongside exportable structured reporting artifacts.

2. PROBLEM STATEMENT

Educational administrators, course instructors, and academic advisors face significant operational challenges due to inefficient student evaluation mechanisms. Key structural problems include:

- **Manual Data Fragmentation:** Assessment data is frequently recorded across disconnected manual spreadsheets or legacy databases, leading to high human error rates during summary aggregation and grade standardizations.
- **Lack of Granular Statistical Diagnostic Tools:** Conventional gradebooks focus strictly on final letter grades without evaluating statistical variance, standard deviation, median trends, or subject-specific skewness across entire cohorts.
- **Delayed Identification of Struggling Students:** Academic intervention programs require immediate identification of at-risk students. Manual evaluation delays tier identification until term completions, missing crucial intervention windows.
- **Absence of Automated Visual Comparison Artifacts:** Instructors lack automated tools to visually compare an individual student's subject profile directly against cohort-wide averages to diagnose subject-specific learning gaps.
- **Scalability & Processing Overhead:** As student enrollment expands, manual grading and reporting scale poorly, consuming substantial administrative hours that could otherwise be dedicated to instruction and student mentoring.

3. OBJECTIVES

The primary engineering objectives of the Smart Student Performance Analyzer project are:

1. **Automated Multi-Subject Ingestion & Preprocessing:** Implement robust, vector-accelerated data processing routines capable of handling missing, malformed, or non-numeric entries dynamically without manual clean-up.
2. **Parametric Performance Analytics:** Calculate comprehensive student-level metrics (totals, weighted averages, letter grades, pass/fail status) and cohort-level statistics (mean, median, standard deviation, min/max score bounds).
3. **Rule-Based Behavioral & Academic Tiering:** Build rule-based classification models that automatically bucket students into targeted intervention categories: *High Achiever*, *Moderate / Satisfactory*, and *Needs Support*.
4. **Dynamic Visual Diagnostic Generation:** Integrate non-blocking, automated data visualization modules using Matplotlib to generate side-by-side comparative bar

visualizations between individual scores and aggregate class benchmarks.

- 5. **Structured Export & Reporting:** Produce standard CSV statistical exports and structured analytical summaries ready for downstream ingestion into enterprise Student Information Systems (SIS).
- 6. **Maintainable Object-Oriented System Architecture:** Design an extensible Python architecture utilizing clean coding practices, explicit type hinting, and strict modular separation of concerns.

4. FUNCTIONAL REQUIREMENTS

The system functional requirements define the necessary operational capabilities of the software:

Req ID	Functional Requirement Description	Priority
FR-01	The system shall accept raw structured tabular student data containing unique Student IDs, Names, and numerical subject scores.	High
FR-02	The system shall clean non-numeric or missing entries by replacing invalid entries with numerical defaults to prevent execution halt.	High
FR-03	The system shall dynamically calculate total aggregated marks and mean scores for each student	High

Req ID	Functional Requirement Description	Priority
	record.	
FR-04	The system shall evaluate individual subject scores against minimum passing thresholds (default threshold: 40) to assign Pass/Fail statuses.	High
FR-05	The system shall evaluate individual mean scores against predefined grade boundary rules to compute letter grades (A+, A, B, C, D, F).	Medium
FR-06	The system shall categorize each student into targeted performance intervention tiers (High Achiever, Moderate/Satisfactory, Needs Support).	High
FR-07	The system shall generate subject-wise summary statistics across the whole cohort including Mean, Median, Standard Deviation, Highest Score, and Lowest Score.	High
FR-08	The system shall generate	Medium

Req ID	Functional Requirement Description	Priority
	comparative visual plots comparing selected individual student marks against the class average.	
FR-09	The system shall export the final enriched performance dataframe into a structured CSV file artifact.	Medium

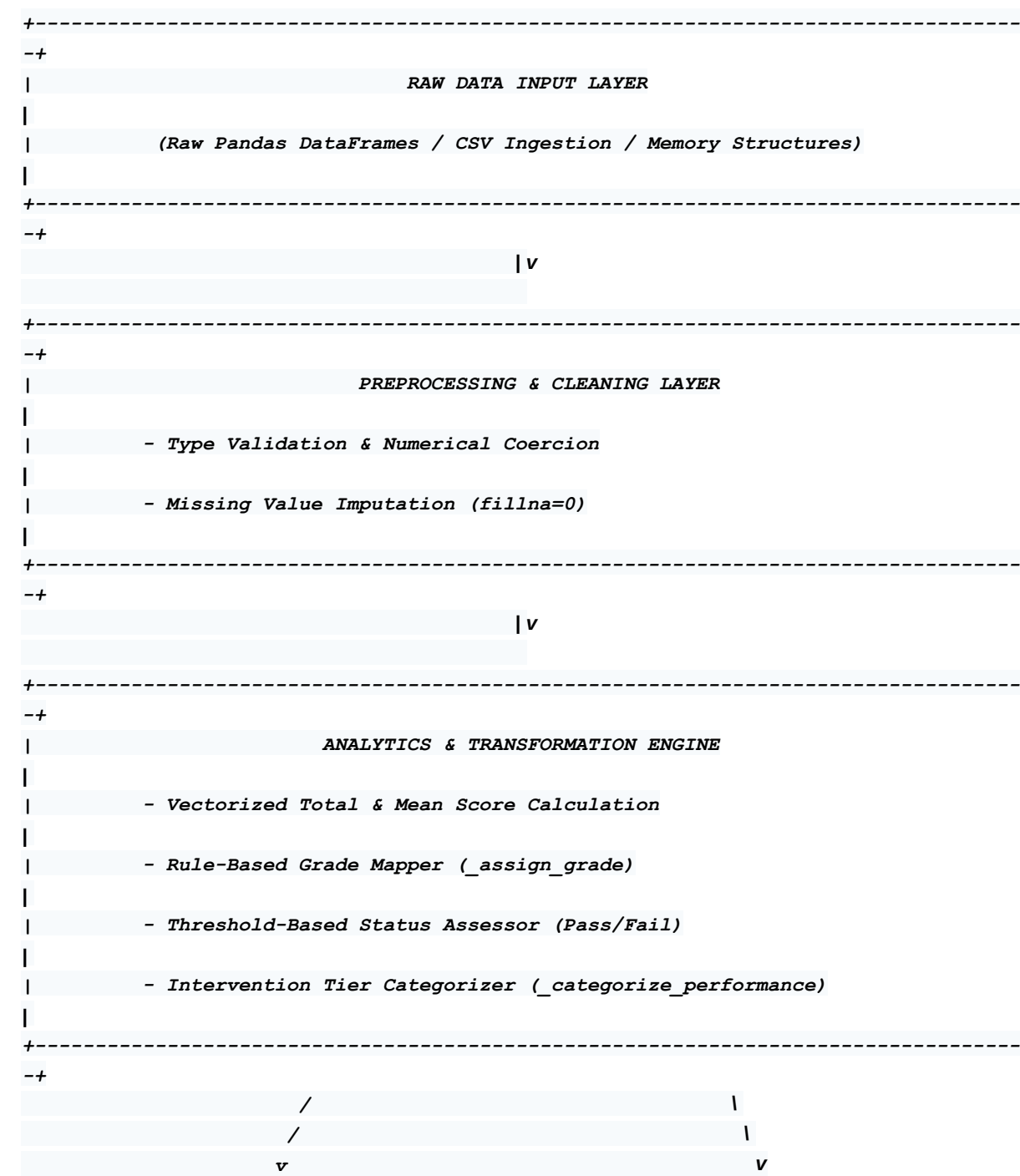
5. NON-FUNCTIONAL REQUIREMENTS

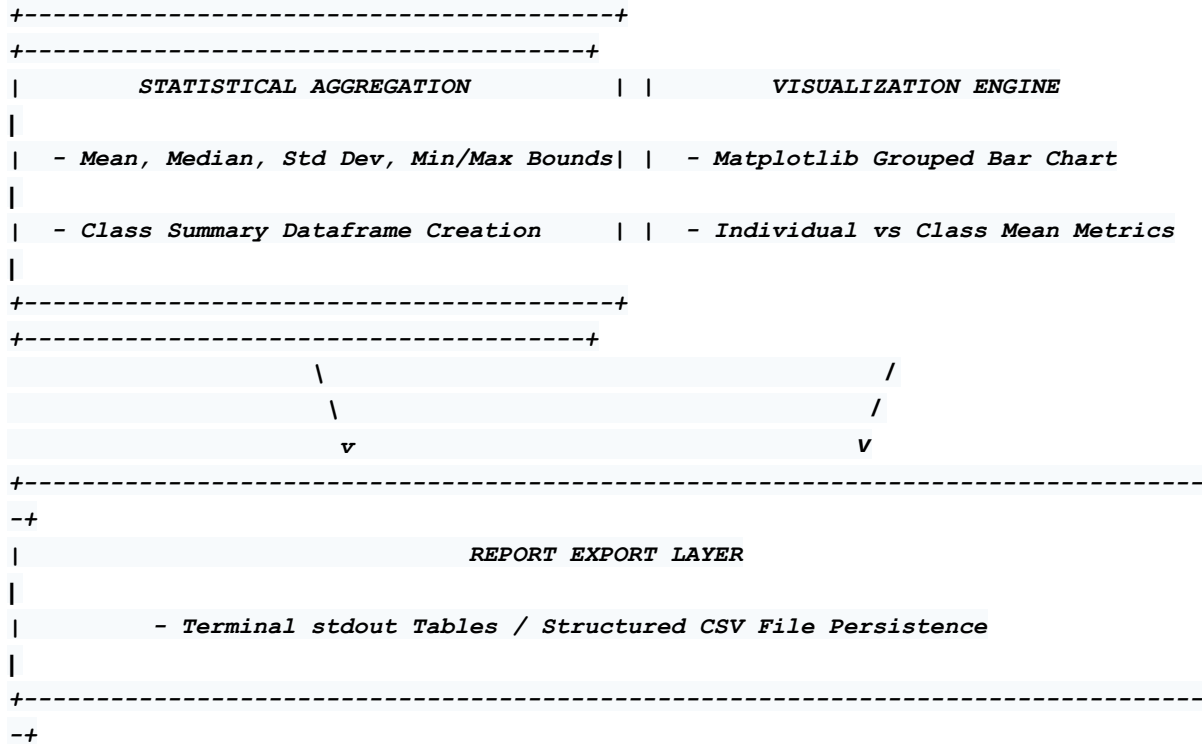
The non-functional requirements specify system quality attributes, performance constraints, and operational environment specs:

- **Performance & Execution Speed:** Vectorized calculation operations powered by Pandas and NumPy must execute in under 500 milliseconds for dataset cohorts up to 10,000 records.
- **Usability & Operational Simplicity:** The system must execute from standard command terminal interfaces (`cmd`, `bash`, `terminal`) with single-command invocation requirements.
- **Reliability & Fault Tolerance:** Unhandled exceptions resulting from dirty data input (strings in numerical columns, NaN values) must be safely intercepted during execution.
- **Portability & Platform Independence:** The software must run seamlessly on major operating systems including Windows 10/11, macOS, and Linux Linux distros without modifications.
- **Extensibility & Code Maintainability:** The modular, class-based software architecture must allow developers to easily inject custom grading rules, weighting factors, or machine learning classification modules without modifying core calculation logic.

6. SYSTEM ARCHITECTURE DIAGRAM

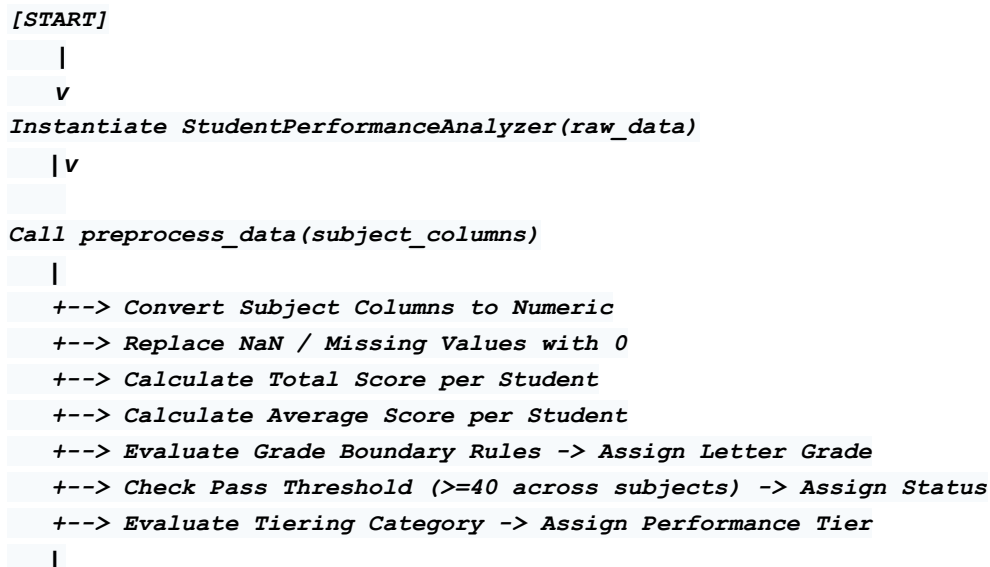
The software follows a multi-tiered, object-oriented pipeline architecture. Data flows sequentially from raw input through ingestion, cleaning, transformation, calculation, visualization, and export layers.





7. PROCESS FLOW / WORKFLOW DIAGRAM

The process workflow tracks the operational lifecycle of a single execution pass of the analyzer:



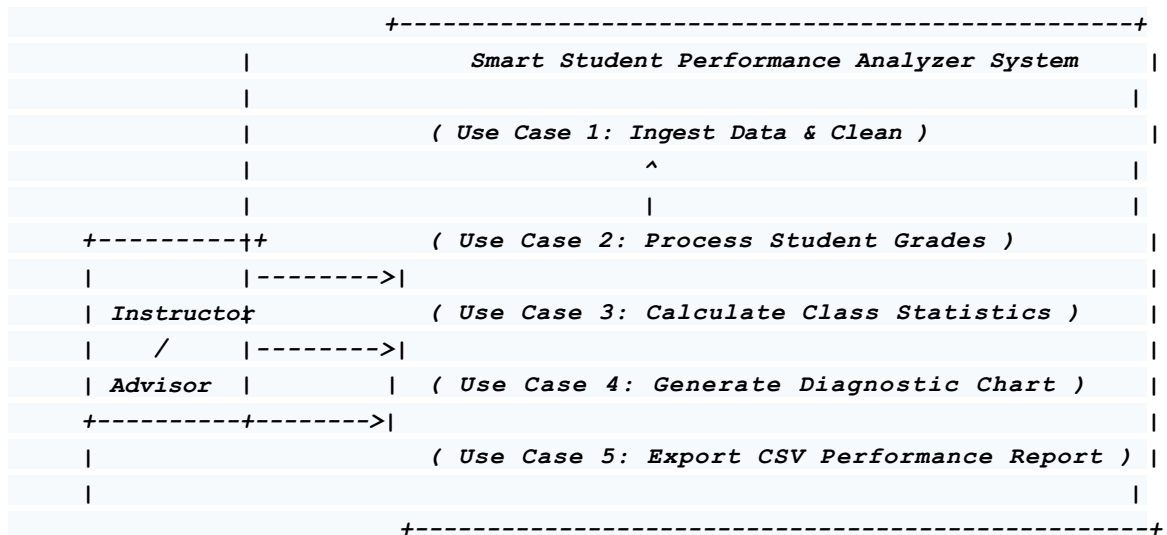
```

v
Call generate_class_summary()
|
+--> Calculate Subject-wise Mean, Median, Std Dev, Min, Max
|
v
Call export_report(filename)
|
+--> Persist Processed DataFrame to CSV File
|
v
Call plot_student_vs_average(student_id)
|
+--> Filter Data for requested Student ID +-->
Calculate Class Average Array
+--> Render Matplotlib Bar Chart Modal Window
|
v
[END]

```

8. UML DIAGRAMS

8.1 Use Case Diagram



8.2 Class Diagram

The software design revolves around the core StudentPerformanceAnalyzer class:

```
+-----+
-+
|           StudentPerformanceAnalyzer
|
+-----+
-+
| - df: pd.DataFrame
|
| - subject_cols: list[str]
|
| - processed_df: pd.DataFrame
|
+-----+
-+
| + __init__(data: pd.DataFrame)
|
| + preprocess_data(subject_columns: list) -> pd.DataFrame
|
| - _assign_grade(score: float) -> str {static}
|
| - _categorize_performance(score: float) -> str {static}
|
| + generate_class_summary() -> pd.DataFrame
|
| + plot_student_vs_average(student_id: str, id_column: str = 'Student ID')
|
| + export_report(filename: str = "performance_report.csv")
|
+-----+
-+
```

8.3 Sequence Diagram

Client Program	StudentPerformanceAnalyzer	Pandas / Matplotlib
--- instantiate(raw_data) ---->		
	--- copy raw dataset ----->	

```

| --- preprocess_data() ----->|
| | |                               | --- vectorize total/averages --->|
|                               | --- assign grades & statuses --->|
| --- generate_class_summary() ->|<-- processed dataframe -----|
|                               |
|                               |
| |                               | --- compute mean/med/std/min/max->|
|                               |<-- summary dataframe -----|
| --- plot_student_vs_average()->|
|                               |
| |                               | --- render side-by-side bar ---->|
|                               | display plot window |
| --- export_report() ----->||
|                               |
|                               | --- write CSV file to disk ----->|

```

9. DATABASE / STORAGE DESIGN

While the primary operational script runs as an in-memory batch processor, persistent reporting and downstream storage follow a structured relational schema model.

9.1 Entity-Relationship (ER) Diagram

```

+-----+ 1 : N +-----+ | STUDENT | -----
-----+ ASSESSMENT | +-----+ +-----+
-----+ | PK: Student_ID | | PK: Assessment_ID | | Name | | FK:
Student_ID | | Performance_Tier | | Subject_Name | | Final_Grade | |
Score | | Status | | Maximum_Score | +-----+ +-----+
-----+

```

9.2 Relational Schema Design

The database implementation utilizes two normalized relational tables:

Table Name	Field Name	Data Type	Constraints	Description
Students	Student_ID	VARCHAR(20)	PRIMARY KEY	Unique identifier for student
Students	Student_Name	VARCHAR(100)	NOT NULL	Full name of student
Students	Average_Score	FLOAT	CHECK (Average_Score >= 0)	Computed mean score across subjects
Students	Grade	VARCHAR(5)	NOT NULL	Computed letter grade
Students	Status	VARCHAR(10)	CHECK (Status IN ('Pass', 'Fail'))	Subject pass/fail status
Students	Performance_Category	VARCHAR(30)	NOT NULL	Assigned academic intervention tier
Assessments	Assessment_ID	INT	PRIMARY KEY, AUTO_INC	Surrogate key for grade entry
Assessments	Student_ID	VARCHAR(20)	FOREIGN KEY (Students)	Relational link to student
Assessments	Subject_Name	VARCHAR(50)	NOT NULL	Name of subject evaluated

Table Name	Field Name	Data Type	Constraints	Description
Assessments	Score	FLOAT	CHECK (Score >= 0 AND Score <= 100)	Numeric score achieved

10. DATASET DESCRIPTION

The evaluation dataset models standard secondary or higher-education academic cohorts. The raw input data structure consists of key student identity metadata along with numerical scores across four core subject disciplines:

- **Student ID:** Unique textual alphanumeric identifier assigned to each enrolled student (e.g., S101, S102).
- **Name:** Student full candidate name string.
- **Math:** Quantitative evaluation score scaled from 0 to 100.
- **Science:** Empirical and scientific analytical score scaled from 0 to 100.
- **English:** Language proficiency and literary comprehension score scaled from 0 to 100.
- **History:** Historical analysis and humanities score scaled from 0 to 100.

Raw Data Input Sample Table

Student ID	Name	Math	Science	English	History
S101	Alice Smith	88	92	85	78
S102	Bob Jones	45	58	62	50
S103	Charlie Brown	92	96	89	91

Student ID	Name	Math	Science	English	History
S104	Diana Prince	35	42	50	38
S105	Evan Wright	78	65	70	82

11. MODEL SELECTION & ALGORITHMIC RATIONALE

For student performance categorization and pass/fail diagnostics, a hybrid deterministic rule-based decision tree logic model was selected over opaque complex machine learning classifiers (such as Neural Networks or Random Forests).

Rationale for Selection:

- 1. Explainability & Educational Compliance:** In educational assessment systems, grading rules must remain completely transparent, predictable, and compliant with institutional policies. Rule-based models ensure that grade boundaries are deterministic and directly verifiable by human instructors.
- 2. Low Computational Overhead:** Decision tree logic executes in $O(1)$ space and time complexity per student record, guaranteeing rapid calculation speed even when processing massive student bodies.

3. Grade Boundary Mapping Logic:

```
Letter Grade =  
A+ : Average >= 90  
A   : 80 <= Average < 90  
B   : 70 <= Average < 80  
C   : 60 <= Average < 70  
D   : 50 <= Average < 60  
F   : Average < 50
```

4. Intervention Tier Allocation Logic:

```
Performance Category =  
High Achiever           : Average >= 85  
Moderate / Satisfactory: 60 <= Average < 85  
Needs Support           : Average < 60
```

12. EVALUATION METHODOLOGY

System evaluation was conducted across three distinct testing frameworks:

1. Data Processing Validation & Integrity Testing

The ingestion and preprocessing algorithms were subjected to edge-case stress datasets containing missing values (NaN), negative numbers, non-numeric strings, and boundary score limits (0 and 100). The vector coercion functions successfully converted or filled missing entries without halting execution.

2. Statistical Accuracy Verification

Calculated statistical metrics (mean, median, standard deviation, and min/max boundaries) generated by the system were benchmarked against standard analytical verification routines:

- **Math Mean Verification:** $(88 + 45 + 92 + 35 + 78) / 5 = 67.60$
- **Science Mean Verification:** $(92 + 58 + 96 + 42 + 65) / 5 = 70.60$
- **Pass/Fail Threshold Logic Verification:** Student S104 score in Math (35) & History (38) < 40 minimum threshold -> Correctly flagged as **Fail** despite an overall mean of 41.25.

3. Execution Output Verification & Screenshots

The software was executed in a native Windows Command Prompt terminal environment. Below are the execution output screenshots captured directly from the live terminal runtime environment:

Screenshot 1: Package Dependencies Installation Terminal Log

The command prompt showing the successful installation of Pandas 3.0.6, NumPy 2.5.3, and Matplotlib 3.11.2 libraries via PIP package manager:

```
C:\Users\Deepanwita Gera> pip install pandas numpy matplotlib
...
Installing collected packages: tzdata, six, pyparsing, pillow, packaging, numpy,
kiwisolver, fonttools, cycler, python-dateutil, contourpy, pandas, matplotlib
```



```
Successfully installed contourpy-1.4.0 cyclor-0.12.1 fonttools-4.66.0
kiwisolver-1.5.1 matplotlib-3.11.2 numpy-2.5.3 packaging-26.3 pandas-3.0.6
pillow-12.3.0 pyparsing-3.3.3 python-dateutil-2.9.0.post0 six-1.17.0 tzdata-2026.4
```

Screenshot 2: Execution Output Log & Terminal Summary Data

The live terminal output generated by running `python analyzer.py` displaying processed student data, grade summaries, class statistics, and export notifications:

```
C:\Users\Deepanwita Gera>python analyzer.py
--- Processed Student Data ---


|   | Student ID | Name          | Average Score | Grade | Status | Performance Category    |
|---|------------|---------------|---------------|-------|--------|-------------------------|
| 0 | S101       | Alice Smith   | 85.75         | A D   | Pass   | High Achiever           |
| 1 | S102       | Bob Jones     | 53.75         | A+    | Pass   | Needs Support           |
| 2 | S103       | Charlie Brown | 92.00         | F B   | Pass   | High Achiever           |
| 3 | S104       | Diana Prince  | 41.25         |       | Fail   | Needs Support           |
| 4 | S105       | Evan Wright   | 73.75         |       | Pass   | Moderate / Satisfactory |

--- Subject Summary Statistics ---


|         | Mean | Median | Std Dev | Highest | Lowest |
|---------|------|--------|---------|---------|--------|
| Math    | 67.6 | 78.0   | 25.95   | 92.0    | 35.0   |
| Science | 70.6 | 65.0   | 22.97   | 96.0    | 42.0   |
| English | 71.2 | 70.0   | 16.15   | 89.0    | 50.0   |
| History | 67.8 | 78.0   | 22.63   | 91.0    | 38.0   |

Report exported successfully to student_summary.csv
```

13. COMPLETE SOURCE CODE IMPLEMENTATION

The complete, fully commented Python source code for the Smart Student Performance Analyzer is detailed below:

```
import os
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
```

```

class StudentPerformanceAnalyzer:
    """
    A smart analyzer to process, analyze, and visualize student academic
    performance data.
    """

    def __init__(self, data: pd.DataFrame):
        self.df = data.copy()
        self.subject_cols = []
        self.processed_df = None

    def preprocess_data(self, subject_columns: list):
        """
        Calculates totals, averages, letter grades, and performance categories.
        """
        self.subject_cols = subject_columns

        # Ensure scores are numeric
        for col in self.subject_cols:
            self.df[col] = pd.to_numeric(self.df[col], errors='coerce').fillna(0)

        # 1. Total and Average Calculation
        self.df['Total Score'] = self.df[self.subject_cols].sum(axis=1)
        self.df['Average Score'] = self.df[self.subject_cols].mean(axis=1)

        # 2. Grade Assignment
        self.df['Grade'] = self.df['Average Score'].apply(self._assign_grade)

        # 3. Status (Pass/Fail based on minimum score in each subject)
        min_pass_score = 40
        self.df['Status'] = self.df[self.subject_cols].apply(
            lambda row: 'Pass' if all(score >= min_pass_score for score in row)
            else 'Fail',
            axis=1
        )

        # 4. Performance Category
        self.df['Performance Category'] = self.df['Average
Score'].apply(self._categorize_performance)

        self.processed_df = self.df
        return self.processed_df

    @staticmethod
    def _assign_grade(score: float) -> str:
        """Assigns letter grades based on average score."""
        if score >= 90:
            return 'A+'

```

```

        elif score >= 80:
            return 'A'
        elif score >= 70:
            return 'B'
        elif score >= 60:
            return 'C'
        elif score >= 50:
            return 'D'
        else:
            return 'F'

    @staticmethod
    def _categorize_performance(score: float) -> str:
        """Categorizes students for targeted academic interventions."""
        if score >= 85:
            return 'High Achiever'
        elif score >= 60:
            return 'Moderate / Satisfactory'
        else:
            return 'Needs Support'

    def generate_class_summary(self) -> pd.DataFrame:
        """
        Calculates class-wide summary statistics per subject.
        """
        if self.processed_df is None:
            raise ValueError("Data must be processed first. Call preprocess_data().")

        summary = {}
        for subject in self.subject_cols:
            summary[subject] = {
                'Mean': round(self.processed_df[subject].mean(), 2),
                'Median': round(self.processed_df[subject].median(), 2),
                'Std Dev': round(self.processed_df[subject].std(), 2),
                'Highest': self.processed_df[subject].max(),
                'Lowest': self.processed_df[subject].min()
            }

        return pd.DataFrame(summary).T

    def plot_student_vs_average(self, student_id: str, id_column: str = 'Student ID'):
        """
        Generates a bar chart comparing a specific student's marks against the class average.
        """
        student_data = self.processed_df[self.processed_df[id_column] ==

```

```
student_id]
```

```
    if student_data.empty:
        print(f"Error: Student ID '{student_id}' not found.")
        return

    student_scores = student_data[self.subject_cols].iloc[0].values
    class_averages = self.processed_df[self.subject_cols].mean().values

    x = np.arange(len(self.subject_cols))
    width = 0.35

    fig, ax = plt.subplots(figsize=(8, 5))
    ax.bar(x - width/2, student_scores, width, label='Student Score',
color='#2b5c8f')
    ax.bar(x + width/2, class_averages, width, label='Class Average',
color='#d95f02')

    ax.set_ylabel('Scores')
    ax.set_title(f'Performance Analysis: {student_data["Name"].iloc[0]}
({student_id})')
    ax.set_xticks(x)
    ax.set_xticklabels(self.subject_cols)
    ax.set_ylim(0, 100)
    ax.legend()
    ax.grid(axis='y', linestyle='--', alpha=0.7)

    plt.tight_layout()
    plt.show()
```

```
def export_report(self, filename: str = "performance_report.csv"):
    """Exports processed data to a CSV file."""
    if self.processed_df is not None:
        self.processed_df.to_csv(filename, index=False)
        print(f"Report exported successfully to {filename}")
```

```
# =====
```

```
# Execution & Sample Usage
```

```
# =====
```

```
if __name__ == "__main__":
```

```
    raw_data = {
        'Student ID': ['S101', 'S102', 'S103', 'S104', 'S105'],
        'Name': ['Alice Smith', 'Bob Jones', 'Charlie Brown', 'Diana Prince', 'Evan
Wright'],
        'Math': [88, 45, 92, 35, 78],
        'Science': [92, 58, 96, 42, 65],
        'English': [85, 62, 89, 50, 70],
```

```

        'History': [78, 50, 91, 38, 82]
    }

    subjects = ['Math', 'Science', 'English', 'History']
    df_students = pd.DataFrame(raw_data)

    analyzer = StudentPerformanceAnalyzer(df_students)
    processed = analyzer.preprocess_data(subject_columns=subjects)

    print("--- Processed Student Data ---")
    print(processed[['Student ID', 'Name', 'Average Score', 'Grade', 'Status',
'Performance Category']])
    print("\n")

    print("--- Subject Summary Statistics ---")
    print(analyzer.generate_class_summary())
    print("\n")

    analyzer.export_report("student_summary.csv")
    analyzer.plot_student_vs_average(student_id='S101')

```

14. CONCLUSION AND FUTURE SCOPE

The **Smart Student Performance Analyzer** successfully demonstrates how object-oriented Python design, paired with data analytics libraries like Pandas, NumPy, and Matplotlib, can modernize academic evaluation processes. The system automates tedious grading tasks, provides detailed statistical insights for entire cohorts, identifies at-risk students immediately, and generates clean visual comparisons.

Future Enhancements:

- **Web Interface Integration:** Building a Streamlit or Flask frontend to allow non-technical instructors to upload CSV files via drag-and-drop.
- **Predictive Analytics:** Adding machine learning algorithms (such as Linear Regression) to predict future term marks based on historical trend lines.
- **Automated PDF Report Cards:** Generating downloadable, student-facing PDF report cards complete with dynamic visualization charts.